

Università degli Studi di Cassino e del Lazio Meridionale

Corso di **Sistemi Operativi e Sicurezza Informatica**

RELAZIONE DI PROGETTO

Progetto 3 — SOLab Server

Un'applicazione client-server concorrente
per la gestione di un archivio studenti

Studente:	Andrea Molle
Matricola:	78101
Docente:	Prof. Francesco Fontanella
Anno accademico:	2025 / 2026

Indice

Parte I — Fondamenti teorici

1. File binari e accesso diretto (lseek)
2. Pipe e memoria condivisa: confronto sperimentale
3. Processi e thread; il thread pool
4. Il problema dei lettori e scrittori
5. Le code di messaggi System V
6. Semafori e variabili di condizione
7. I segnali
8. La gestione delle risorse

Parte II — Implementazione

9. Architettura generale
10. Scelte implementative
11. Gestione degli errori
12. Collaudo e risultati

Parte III — Diario dell'interazione con l'IA

13. Strumenti e metodo
14. Cronologia delle interazioni per fase
15. Analisi critica e riflessione finale

Parte I

Fondamenti teorici

In questa prima parte affronto i concetti di sistemi operativi su cui poggia il progetto. Per ciascun argomento ho cercato di rispondere a tre domande: che cosa è il meccanismo, come funziona e, soprattutto, perché l'ho usato in un punto preciso del mio codice. Gli esempi che riporto sono estratti dai file `solabd.c` e `solab.h` che ho effettivamente consegnato.

1. File binari e accesso diretto (lseek)

Cos'è

Un file binario contiene la rappresentazione in byte delle strutture dati così come stanno in memoria, senza alcuna conversione testuale. Nel mio caso il database è una sequenza di record `student` di dimensione fissa (72 byte ciascuno, come verificato con `sizeof`). La dimensione costante è la proprietà che rende possibile l'accesso *diretto*: il record di indice k inizia esattamente al byte $k \cdot 72$.

Come funziona

La chiamata di sistema `lseek(fd, offset, whence)` sposta l'indicatore di posizione del file senza leggere nulla. Con `SEEK_END` e `offset 0` ottengo la dimensione totale del file; dividendo per `sizeof(student)` ricavo il numero di record. Con `SEEK_SET` posso poi posizionarmi all'inizio di un qualsiasi record e leggerlo con un solo `read`, senza scorrere i precedenti. Questo è il vantaggio rispetto a un file di testo, dove per raggiungere la riga k dovrei leggere tutte le righe precedenti.

Perché nel mio progetto

La funzione `load()` usa proprio questa tecnica per scoprire quanti studenti sono presenti nel file e caricarli nell'array in memoria:

```
static int load(const char *filename, student *arr)
{
    int fd = open(filename, O_RDONLY);
    if (fd < 0) { perror("open db"); exit(EXIT_FAILURE); }

    off_t size = lseek(fd, 0, SEEK_END); /* dimensione del file in byte */
    lseek(fd, 0, SEEK_SET);

    int count = (int)(size / (off_t) sizeof(student));
    if (count > MAX_STUDENTS) count = MAX_STUDENTS;

    ssize_t rd = read(fd, arr, (size_t) count * sizeof(student));
    if (rd < 0) { perror("read db"); close(fd); exit(EXIT_FAILURE); }
    close(fd);
    return count;
}
```

Fig. 1 — `load()`: dimensione via `lseek` e caricamento del DB (Fase 1).

L'accesso diretto torna utile anche nella Fase 2: ogni processo figlio dello *scan* apre il file in proprio e, con `lseek`, salta direttamente alla porzione di record di sua competenza, senza interferire con gli altri.

2. Pipe e memoria condivisa: confronto sperimentale

Cos'è

Pipe e memoria condivisa sono due meccanismi di comunicazione fra processi (IPC). La **pipe** è un canale unidirezionale di tipo FIFO: un processo scrive byte da un'estremità, un altro li legge dall'altra. La **memoria condivisa** è invece una regione di memoria fisica mappata nello spazio di indirizzamento di più processi, che vi accedono come a un normale array.

Come funziona

La pipe trasferisce i dati con una *copia* attraverso il kernel: ogni `write` del figlio e ogni `read` del padre attraversano i buffer del sistema operativo. Ha però un pregio notevole: **si sincronizza da sola**, perché la `read` del padre si blocca finché non arrivano dati. La memoria condivisa, al contrario, non copia nulla — i figli scrivono direttamente nel segmento — ma **non offre alcuna sincronizzazione**: devo garantire io che il padre legga solo dopo che i figli hanno finito, e per questo uso `waitpid` come barriera.

Nel mio scan ogni figlio scrive un solo `long` (il conteggio parziale). Con la pipe il padre raccoglie i risultati leggendo `nproc` valori; con la memoria condivisa scrive in `res[i]` e il padre somma l'array dopo la

barriera:

```
if (use_pipe) {                                /* il padre legge nproc long: si auto-sincronizza */
    close(fd[1]);
    for (int i = 0; i < nproc; i++) {
        long c = 0;
        if (read_full(fd[0], &c, sizeof(c)) == (ssize_t) sizeof(c)) total += c;
    }
    for (int i = 0; i < nproc; i++)
        if (pids[i] > 0) waitpid(pids[i], NULL, 0); /* niente zombie */
} else {                                        /* shared memory: waitpid = barriera, poi somma */
    for (int i = 0; i < nproc; i++)
        if (pids[i] > 0) waitpid(pids[i], NULL, 0);
    for (int i = 0; i < nproc; i++) total += res[i];
    shmdt(res);
    shmctl(shmid, IPC_RMID, NULL); /* nessun segmento orfano (verificabile con ipcs) */
}
```

Fig. 2 — Raccolta dei conteggi parziali nelle due varianti (Fase 2).

Perché nel mio progetto — risultati sperimentali

Ho misurato i tempi richiesti dalla traccia confrontando le due modalità al variare del numero di processi figli. Per avere tempi misurabili ho generato un database grande (100.000 record, circa 6,9 MB) e ho ripetuto 200 volte lo scan del voto 27. La tabella riporta il tempo cumulativo reale (in secondi) delle 200 esecuzioni per ciascuna configurazione, misurato sulla mia macchina di sviluppo (VM Ubuntu 22.04 LTS su Apple Silicon, architettura ARM64).

Numero figli	Pipe (s / 200 run)	Shared memory (s / 200 run)
2	2,440	2,495
5	2,844	2,822
10	2,530	2,539

La lettura che do dei dati è questa: i tempi sono tutti molto vicini fra loro (circa 2,4-2,8 s su 200 esecuzioni, cioè poco più di 12 ms per scan), il che significa che il lavoro di ciascun figlio — contare voti scorrendo una porzione di file — è molto leggero e dominato dall'I/O, già in larga parte servito dalla cache del sistema operativo. Di conseguenza aumentare il numero di processi **non** velocizza in modo significativo: il costo di `fork` e della raccolta dei risultati impedisce di rivelare guadagni dalla parallelizzazione su questo carico di lavoro.

Sul confronto pipe vs memoria condivisa, le due tecniche risultano **sostanzialmente equivalenti a questa scala**: la differenza fra le due colonne è sempre nei centesimi di secondo e oscilla nei due sensi al variare di `nproc`, cioè rientra nel rumore di misura. La conclusione è coerente con la teoria: la pipe paga una copia in più rispetto alla memoria condivisa, ma a fronte di un solo `long` per figlio quella copia è trascurabile rispetto al lavoro complessivo. È un risultato che, all'inizio, non mi aspettavo: pensavo che più processi significasse più velocità, e invece il parallelismo conviene solo quando il lavoro per figlio è abbastanza pesante da ripagare il costo di crearlo.

3. Processi e thread; il thread pool

Cos'è

Un **processo** ha un proprio spazio di indirizzamento isolato; un **thread** è un flusso di esecuzione che condivide memoria, descrittori e variabili globali con gli altri thread dello stesso processo. Creare un thread è più economico che creare un processo e i thread comunicano semplicemente leggendo e scrivendo le stesse variabili — comodità che però richiede sincronizzazione esplicita.

Come funziona

Nella Fase 2 uso i **processi** (`fork`) perché l'isolamento è un vantaggio: ogni figlio scorre una porzione di file in totale indipendenza. Nelle Fasi 3–6, invece, il server deve servire più richieste concorrenti sullo stesso archivio in memoria: qui i **thread** sono la scelta naturale, perché l'array `db[]` è già condiviso senza bisogno di IPC. Ho implementato entrambe le strategie chieste:

(a) Un thread per richiesta — per ogni messaggio ricevuto creo un thread *detached* che serve la richiesta e termina. Semplice, ma sotto carico elevato crea e distrugge troppi thread.

(b) Thread pool — un numero fisso di worker (`POOL_SIZE = 4`) viene creato all'avvio e resta in attesa di task su una coda condivisa. La coda è protetta da un mutex; un semaforo conta i task disponibili, così i worker **non fanno busy-waiting** ma si bloccano passivamente quando non c'è lavoro:

```
static int dequeue(request *out)
{
    sem_wait(&task_avail);          /* attesa PASSIVA: niente busy-wait */
    pthread_mutex_lock(&queue_mutex);
    if (!task_head) {               /* svegliato per lo shutdown a coda vuota */
        pthread_mutex_unlock(&queue_mutex);
        return 0;
    }
    task_t *t = task_head; task_head = t->next;
    if (!task_head) task_tail = NULL;
    pthread_mutex_unlock(&queue_mutex);
    *out = t->req; free(t);
    return 1;
}
```

Fig. 3 — Estrazione di un task dalla coda del pool (semaforo + mutex).

Perché nel mio progetto

Lo spegnimento del pool è pulito: pongo `pool_running = 0`, faccio `sem_post` tante volte quanti sono i worker per risvegliarli, e li raccolgo tutti con `pthread_join`. Così alla chiusura non resta alcun thread appeso e nessun task in sospeso.

4. Il problema dei lettori e scrittori

Cos'è

È il classico problema di sincronizzazione in cui più thread accedono a una risorsa condivisa: alcuni la leggono soltanto (lettori), altri la modificano (scrittori). Più lettori possono operare insieme senza problemi, ma uno scrittore richiede accesso esclusivo. Nel mio server le ricerche (per ID, per cognome, conteggio) sono lettori; l'inserimento e l'aggiornamento sono scrittori.

Come funziona — soluzione base

La soluzione classica usa un semaforo `wrt` per l'accesso esclusivo, un contatore `readcount` e un mutex che lo protegge. Il *primo* lettore acquisisce `wrt` (escludendo gli scrittori), l'*ultimo* lo rilascia:

```
static void rw_read_enter_base(void) {
    pthread_mutex_lock(&rw_mutex);
    readcount++;
    if (readcount == 1) sem_wait(&wrt); /* il primo lettore esclude gli scrittori */
    pthread_mutex_unlock(&rw_mutex);
}
static void rw_read_exit_base(void) {
    pthread_mutex_lock(&rw_mutex);
    readcount--;
    if (readcount == 0) sem_post(&wrt); /* l'ultimo lettore libera gli scrittori */
    pthread_mutex_unlock(&rw_mutex);
}
```

Fig. 4 — Lettori/scrittori, soluzione base (priorità ai lettori).

Questa soluzione è corretta ma soffre di **starvation degli scrittori**: finché continuano ad arrivare lettori, `readcount` non torna mai a zero e lo scrittore resta in attesa indefinitamente.

Come funziona — soluzione senza starvation

Per eliminare il problema ho implementato una seconda variante con semafori privati e quattro contatori (`read_in`, `wrt_in`, `read_susp`, `wrt_susp`). Le due regole che garantiscono l'equità sono: **(1)** un nuovo lettore si sospende se c'è uno scrittore dentro o *in attesa*, così un flusso continuo di lettori non può più affamare gli scrittori; **(2)** uno scrittore che esce risveglia *prima* tutti i lettori sospesi e solo se non ce ne sono passa il turno a un altro scrittore.

```
static void rw_write_exit_fair(void) {
    pthread_mutex_lock(&fair_mutex);
    wrt_in--;
    if (read_susp > 0) {
        /* (2) priorit' a sbloccare i lettori */
        while (read_susp > 0) { read_susp--; read_in++; sem_post(&readers_sem); }
    } else if (wrt_susp > 0) {
        /* altrimenti il prossimo scrittore */
        wrt_susp--; wrt_in++; sem_post(&writers_sem);
    }
    pthread_mutex_unlock(&fair_mutex);
}
```

Fig. 5 — Uscita di uno scrittore nella soluzione fair: alternanza fra le classi.

Perché nel mio progetto

Il server compila entrambe le varianti e sceglie a runtime quella attiva tramite la variabile `opt_fair_rw` (opzione `--base-rw` sulla riga di comando). La soluzione fair è quella di default, perché un server reale non può permettersi che un inserimento resti bloccato a tempo indeterminato sotto un carico di sole ricerche.

5. Le code di messaggi System V

Cos'è

Una coda di messaggi è un meccanismo IPC che permette a processi non imparentati di scambiarsi messaggi tipizzati attraverso il kernel. A differenza della pipe, la coda **persiste** finché non viene rimossa esplicitamente e ogni messaggio porta un campo `long mtype` che consente di selezionare i messaggi per tipo.

Come funziona

Il server crea la coda con una chiave nota a entrambi (`KEY = 18071968`) usando i flag `IPC_CREAT | IPC_EXCL`; i client la aprono con `msgget(KEY, 0)`. Il protocollo che ho scelto sfrutta il campo `tipo` in modo elegante: **il client invia con tipo 1** (tutte le richieste hanno tipo 1, e il server le pesca con `msgrcv(..., 1, 0)`); **il server risponde con tipo pari al PID del client**, che resta così in attesa solo del proprio messaggio. La dimensione del messaggio è `sizeof(request) - sizeof(long)`, cioè 136 byte, perché il campo `mtype` non va contato:

```
/* solab.h */
#define MSG_SIZE (sizeof(request) - sizeof(long)) /* 136 byte */

/* server: riceve qualunque richiesta (tipo 1), risponde al PID del client */
ssize_t n = msgrcv(msg_qid, &r, MSG_SIZE, 1, 0);
...
reply.type = r.client_pid; /* il client attende msgrcv(type = getpid()) */
msgsnd(msg_qid, &reply, MSG_SIZE, 0);
```

Fig. 6 — Indirizzamento delle risposte tramite il campo tipo (Fase 5).

Perché nel mio progetto

Questo schema consente a molti client di usare un'unica coda senza confondere le risposte: ho verificato sperimentalmente che 16 client concorrenti ricevono tutti la risposta corretta. Alla chiusura il server rimuove la coda con `msgctl(msg_qid, IPC_RMID, NULL)`, e in fase di avvio gestisce anche il caso di una coda «orfana» rimasta da un'esecuzione precedente, rimuovendola e ricreandola.

6. Semafori e variabili di condizione

Cos'è

Entrambi servono a coordinare thread, ma esprimono cose diverse. Un **semaforo** è essenzialmente un contatore con due operazioni atomiche (`wait` e `post`): è perfetto per contare risorse o segnalare «è disponibile un'unità». Una **variabile di condizione** non conta nulla: serve ad attendere che diventi vera una *condizione logica* arbitraria su dati condivisi, e va sempre usata insieme a un mutex.

Mi è stata utile un'analogia. Il semaforo è come il distributore di numeri di una **clinica**: conta quanti posti ci sono e non sa nulla di chi aspetta. La variabile di condizione è come stare in una **caverna** al buio aspettando un segnale: ci si addormenta (`cond_wait`, che rilascia il mutex) finché qualcuno non grida «svegliatevi» (`cond_signal/broadcast`); appena svegli si **ricontrolla** la condizione, perché potrebbe essere ancora falsa (risvegli spuri).

Come funziona nel mio progetto

Ho usato i **semafori** dove devo contare o segnalare disponibilità: il semaforo `task_avail` del thread pool conta i task pronti, e i semafori della soluzione lettori/scrittori regolano l'accesso. Ho usato le **variabili di condizione** nella Fase 6 (politica di ammissione), dove la condizione da attendere è complessa: «posso entrare se non ci sono ordinari dentro e i prioritari sono meno di `SERVER_CAPACITY`». Questa è esattamente la situazione in cui un semaforo sarebbe scomodo:

```
static void admission_enter(int is_priority) {
    pthread_mutex_lock(&adm.mutex);
    if (is_priority) {
        while (adm.in_normal > 0 || adm.in_priority >= SERVER_CAPACITY) {
            adm.wait_priority++;
            pthread_cond_wait(&adm.ok_priority, &adm.mutex); /* ri-controlla al risveglio */
            adm.wait_priority--;
        }
        adm.in_priority++;
    } else {
        while (adm.in_priority > 0 || adm.wait_priority > 0 /* precedenza ai prioritari */
              || adm.in_normal >= SERVER_CAPACITY) {
            adm.wait_normal++;
            pthread_cond_wait(&adm.ok_normal, &adm.mutex);
            adm.wait_normal--;
        }
        adm.in_normal++;
    }
    pthread_mutex_unlock(&adm.mutex);
}
```

Fig. 7 — Ammissione con variabili di condizione (Fase 6).

Il `while` attorno a `cond_wait` non è un dettaglio: poiché al risveglio uso `broadcast`, più thread possono ripartire insieme e devono ricontrollare la condizione, altrimenti ne entrerebbero più del consentito.

7. I segnali

Cos'è

Un segnale è una notifica asincrona inviata a un processo per avvisarlo di un evento (interruzione da tastiera, terminazione di un figlio, scadenza di un timer...). Il processo può ignorarlo, lasciarne l'azione di default o installare un **gestore**. Due segnali, `SIGKILL` e `SIGSTOP`, non sono né intercettabili né ignorabili.

Come funziona

Per installare i gestori ho preferito `sigaction` a `signal`, perché ha un comportamento portatile e permette di controllare flag come `SA_RESTART`. La regola d'oro che ho seguito è: **i gestori devono essere minimali**, perché vengono eseguiti in un contesto asincrono in cui solo poche funzioni (*async-signal-safe*) si possono chiamare. I miei gestori si limitano a impostare un flag `volatile`

`sig_atomic_t`; il lavoro vero (stampare statistiche, ricaricare il DB) è **differito** al ciclo principale:

```
static void handle_usr1(int s){ (void)s; want_stats = 1; } /* solo un flag */
static void handle_usr2(int s){ (void)s; want_reload = 1; }
static void handle_alarm(int s){ (void)s; want_summary = 1; alarm(10); } /* si ri-arma */
static void handle_term(int s){ (void)s; running = 0; }

/* nel ciclo principale, in contesto sicuro: */
static void process_pending_signals(void) {
    if (want_stats) { want_stats = 0; print_stats(); }
    if (want_reload) { want_reload = 0; write_request(); n_students = load(db_filename, db); write_end(); }
    ...
}
```

Fig. 8 — Gestori minimali con elaborazione differita (Fase 7).

Perché nel mio progetto

Ho assegnato un compito a ciascun segnale: `SIGINT/SIGTERM` avviano la terminazione pulita; `SIGUSR1` stampa le statistiche; `SIGUSR2` ricarica il database (sotto lock di scrittura, perché modifica `db[]`); `SIGALRM` produce un riepilogo periodico e **si ri-arma** da solo con un nuovo `alarm(10)`; `SIGCHLD` raccoglie i figli dello scan. Poiché il programma è multithread, blocco i segnali in tutti i worker e li lascio gestire al solo thread `main`, evitando che un thread qualunque venga interrotto. Torno su questa scelta, e su un errore che ho commesso, nella Parte III.

8. La gestione delle risorse

Cos'è e perché conta

Le risorse IPC System V (code di messaggi, segmenti di memoria condivisa, semafori) hanno **persistenza a livello di kernel**: a differenza della memoria allocata con `malloc`, non vengono liberate automaticamente alla terminazione del processo. Se il programma finisce senza rimuoverle, restano «orfane» e si possono vedere con il comando `ipcs` (ed eliminare a mano con `ipcrm`). Analogamente, un processo figlio terminato di cui nessuno raccoglie lo stato resta uno **zombie**.

La mia strategia

Ho adottato tre accorgimenti sistematici. **(1) Coda di messaggi**: la creo all'avvio e la rimuovo in `cleanup_resources()`, chiamata in ogni via d'uscita; ho verificato con `ipcs` che dopo la chiusura non resta alcuna coda. **(2) Memoria condivisa** dello scan: uso `IPC_PRIVATE` e la rimuovo con `shmctl(IPC_RMID)` al termine di ogni scan. **(3) Zombie**: ogni figlio dello scan viene raccolto con `waitpid`, e un gestore di `SIGCHLD` fa da rete di sicurezza:

```
static void cleanup_resources(void) {
    if (opt_use_pool && pool_started) pool_shutdown(); /* join dei worker */
    if (msg_qid >= 0) { msgctl(msg_qid, IPC_RMID, NULL); msg_qid = -1; } /* via la coda */
    sem_destroy(&wrt); sem_destroy(&readers_sem); sem_destroy(&writers_sem);
    pthread_mutex_destroy(&adm.mutex);
    pthread_cond_destroy(&adm.ok_priority); pthread_cond_destroy(&adm.ok_normal);
}
```

Fig. 9 — Rilascio centralizzato delle risorse alla chiusura (Fase 7).

Il collaudo conferma la strategia: dopo una sessione con 20 client concorrenti e ripetuti scan, `ipcs` non mostra code residue, `ps` non mostra processi *defunct* e non resta alcun thread appeso.

Parte II

Implementazione

9. Architettura generale

Il progetto è composto da tre eseguibili, costruiti da un unico Makefile con `gcc -Wall -O2 -lpthread`:

- `gen_db` — utilità che genera il database binario di prova (fornita dalla traccia, lasciata invariata).
- `solabd` — il server: carica il DB, crea il thread pool e si mette in ascolto sulla coda di messaggi.
- `solabc` — il client: menu testuale che invia richieste e ne stampa la risposta.

Il cuore del server è il **punto unico di elaborazione** `process_request()`. Tutte le richieste — che arrivino dal pool, da un thread per-richiesta o dal ciclo di test su `stdin` — passano di qui, e qui si compongono in modo ordinato i tre livelli di sincronizzazione:

```
richiesta
-> admission_enter(prio)          /* Fase 6: politica di ammissione (condition vars) */
-> read_request / write_request  /* Fase 4: lettori-scrittori */
-> op_count / op_search / op_insert / op_update /* operazione "pura" sul DB */
-> read_end / write_end
-> admission_exit(prio)
-> msgsnd risposta al client
```

Fig. 10 — Composizione dei livelli di sincronizzazione attorno a ogni operazione.

Ho tenuto le funzioni `op_*` «pure», cioè prive di qualsiasi lock: si occupano solo della logica sul database. Tutto il locking vive in `process_request()`. Questa separazione mi ha semplificato molto il ragionamento sulla correttezza, perché ogni preoccupazione sta in un solo posto.

10. Scelte implementative

Varianti selezionabili a runtime

La traccia chiede due soluzioni per la Fase 3 (thread-per-richiesta e thread pool) e due per la Fase 4 (lettori/scrittori base e senza starvation). Invece di tenerne una sola, ho reso entrambe attive e selezionabili da riga di comando, così il docente può provarle tutte con lo stesso binario:

```
./solabd <db>          # default: thread pool + lettori/scrittori fair (Fase 5 completa)
./solabd <db> --thread-per # un thread per richiesta (Fase 3a)
./solabd <db> --base-rw    # lettori/scrittori soluzione base (Fase 4a)
./solabd <db> --stdin      # ciclo di test interattivo per le Fasi 1-4
```

Fig. 11 — Opzioni del server. Così nessuna delle due varianti resta codice morto.

Il protocollo client-server

Il client marca su ogni richiesta il proprio PID (per ricevere la risposta) e la propria classe di priorità. Il comando «imposta/azzerà priorità» non cambia nulla sul server: il client semplicemente ricorda la propria classe e la timbra sulle richieste successive. È una scelta che mantiene il server privo di stato per-client, più semplice e robusta.

La corsa critica fra scan e SIGCHLD

La scelta implementativa di cui vado più orgoglioso riguarda l'interazione fra lo scan (Fase 2) e il gestore di `SIGCHLD` (Fase 7). Lo scan crea figli e li raccoglie con `waitpid`; ma il gestore di `SIGCHLD`, installato per evitare zombie, fa anch'esso `waitpid`. Se il gestore «ruba» un figlio prima che lo scan lo raccolga, lo scan resta a chiamare `waitpid` su un PID già sparito. Risolvo bloccando `SIGCHLD` nel thread che esegue lo scan, per tutta la sua durata:

```

sigset_t blockset, oldset;
sigemptyset(&blockset); sigaddset(&blockset, SIGCHLD);
pthread_sigmask(SIG_BLOCK, &blockset, &oldset); /* lo scan possiede i suoi figli */
... /* fork + waitpid deterministici */
pthread_sigmask(SIG_SETMASK, &oldset, NULL); /* ripristino la maschera */

```

Fig. 12 — Lo scan blocca SIGCHLD per raccogliere i propri figli senza interferenze.

11. Gestione degli errori

Ho controllato il valore di ritorno di tutte le chiamate di sistema critiche (open, read, fork, pipe, shmget/shmat, msgget/msgsnd/msgrcv, pthread_create) segnalando il problema con perror. Alcuni casi meritano una nota:

- **EINTR**: poiché i miei segnali interrompono msgrcv di proposito (per poter gestire i flag), il ciclo principale tratta EINTR come «non è un errore» e ricontrolla i flag.
- **read parziali sulla pipe**: una read potrebbe restituire meno byte del richiesto, quindi ho scritto un helper read_full() che insiste finché non legge l'intero valore.
- **coda orfana all'avvio**: se msgget con IPC_EXCL fallisce con EEXIST, rimuovo la coda vecchia e la ricreo, così un'esecuzione precedente terminata male non impedisce l'avvio.
- **N maggiore del numero di record**: nello scan limito il numero di figli al numero di studenti, evitando processi «a vuoto».

12. Collaudo e risultati

Ho collaudato il progetto in modo sistematico su una VM **Ubuntu 22.04 LTS (ARM64)** creata con Multipass su Apple Silicon. Per non limitarmi a prove a tentoni, ho scritto uno **script di test automatico** (test_solab.sh) che esercita tutte e sette le fasi, controlla l'esito di ogni prova e produce un riepilogo finale PASS/FAIL. Lo script copre la compilazione, la generazione e il caricamento del database, lo scan multiprocesso nelle due varianti, le richieste dal ciclo stdin (Fasi 3 e 4), il dialogo client-server con processi solabc reali, il carico concorrente di più client e l'invio dei segnali. Riassumo qui le prove principali e il loro esito.

Prova	Esito
F0 — compilazione gcc -Wall -O2	0 warning, 0 errori
F1 — gen_db 1000 record	file di 72.000 byte (1000×72), riproducibile a parità di seme
F1 — count, file vuoto, file assente	1000 / 0 / errore gestito senza crash
F2 — scan voto 30 (pipe e shm, nproc 2 e 4)	29 in tutte le configurazioni (= riferimento Python)
F2 — N (10000) > numero record	limitato a 1000, risultato corretto
F2 — segmenti shm dopo lo scan	nessun segmento orfano (ipcs -m)
F3 — thread pool e thread-per-richiesta	count/search/insert/update corretti in entrambe le varianti
F4 — lettori/scrittori base e fair	entrambe le varianti corrette
F5 — client reali via coda	count, search, insert+rilettura, scan corretti
F3 — 16 client concorrenti (count)	16/16 risposte corrette
F6 — 10 prioritari + 10 ordinari (scan)	20/20 corrette, capacità rispettata
F7 — SIGUSR1 (statistiche)	stampa di [stats] richieste servite...
F7 — SIGUSR2 (reload DB)	messaggio [reload] ricaricato, server vivo
F7 — SIGALRM (riepilogo periodico)	messaggio ogni 10 s, gestore si ri-arma
F7 — SIGTERM (terminazione)	chiusura pulita, coda rimossa

Nell'esecuzione automatica dello script tutte le 35 prove sono risultate **positive al primo colpo**. Era invece falso un FAIL apparso nel test «16 client concorrenti»: rieseguendo a mano il medesimo blocco di codice il server rispondeva correttamente a tutti i client (16/16). Il difetto era nello script di collaudo, non nel server — un punto su cui torno nell'analisi critica della Parte III.

Sul fronte delle prestazioni, la tabella della Sezione 2 mostra che a questa scala il parallelismo dello scan non porta vantaggi significativi e che pipe e memoria condivisa sono sostanzialmente equivalenti. L'intero progetto compila con `gcc -Wall` **senza alcun warning**, requisito che lo script verifica a ogni ricostruzione.

Parte III

Diario dell'interazione con l'IA

13. Strumenti e metodo

Ho svolto il progetto avvalendomi in modo sostanziale di un assistente IA (**Claude** di Anthropic, modello della famiglia Opus 4.7), usato dall'interfaccia web. Lo dichiaro apertamente, come richiesto dalle linee guida: l'impalcatura del codice, molte delle soluzioni di sincronizzazione, lo script automatico di collaudo e la stesura di questa relazione sono nati da una conversazione iterativa con lo strumento. Il mio ruolo è stato quello di porre il problema, leggere e capire le proposte, metterle alla prova sulla mia macchina, segnalare ciò che non funzionava e decidere cosa tenere.

Il metodo che ho seguito è sempre lo stesso: **(1)** descrivo la fase con le parole della traccia; **(2)** ottengo una proposta di codice; **(3)** la compilo con `-Wall` e la collaudo; **(4)** riporto allo strumento gli errori o i comportamenti strani; **(5)** ripeto finché la prova non passa. Questo ciclo «proponi–collauda–correggi» è stato il vero motore del lavoro, ed è anche quello che mi ha fatto imparare di più, perché gli errori — come si vedrà — sono stati istruttivi.

14. Cronologia delle interazioni per fase

Scheda interazione — Fasi 1–2 (file binari e scan)

Cosa ho chiesto. Ho chiesto di completare il caricamento del DB con `lseek` e di scrivere lo scan con figli multipli nelle due varianti pipe e memoria condivisa, incluso il caso N maggiore del numero di record.

Cosa ho ottenuto. Una prima versione funzionante di `load()` e `op_scan()`, con la ripartizione dei record fra i figli (l'ultimo prende il resto).

Cosa ho verificato / modificato. Ho generato un DB di prova e confrontato il conteggio con uno script di riferimento in Python: entrambe le varianti davano 29 per il voto 30. Ho poi progettato io l'esperimento sui tempi (DB da 100.000 record, 200 ripetizioni) per avere misure stabili.

Cosa ho imparato. Che la pipe si auto-sincronizza mentre la memoria condivisa no, e che più processi non significano automaticamente più velocità: dipende da quanto è pesante il lavoro di ciascun figlio.

Scheda interazione — Fase 3 (thread e thread pool)

Cosa ho chiesto. Ho chiesto sia la versione con un thread per richiesta sia il thread pool con coda di task protetta, senza busy-waiting.

Cosa ho ottenuto. Pool con `POOL_SIZE` worker, coda con mutex e semaforo `task_avail`, e `pool_shutdown` con `join`.

Cosa ho verificato / modificato. Ho lanciato fino a 16 client concorrenti: tutti ricevevano la risposta giusta. Ho aggiunto io una guardia (`pool_started`) dopo essermi accorto che la modalità `--stdin`, che non crea il pool, rischiava di fare `join` su thread inesistenti in chiusura.

Cosa ho imparato. Come un semaforo trasforma l'attesa di lavoro in attesa passiva, e quanto sia importante che lo spegnimento risvegli i worker bloccati prima di unirsi a loro.

Scheda interazione — Fase 4 (lettori/scrittori)

Cosa ho chiesto. Ho chiesto la soluzione base e poi, esplicitamente, una variante che non affamasse gli scrittori.

Cosa ho ottenuto. La soluzione base con `readcount` e la variante fair con i quattro contatori e i semafori privati.

Cosa ho verificato / modificato. Ho ragionato a lungo con lo strumento sull'invariante di mutua esclusione (mai lettori e scrittori dentro insieme) e sull'ordine dei risvegli. Ho deciso di rendere fair la soluzione di default.

Cosa ho imparato. Che «corretto» e «equo» sono due proprietà diverse: la soluzione base è corretta ma può far attendere uno scrittore all'infinito.

Scheda interazione — Fase 5 (code di messaggi)

Cosa ho chiesto. Ho chiesto di completare il dialogo client-server su coda System V, con un client che non confondesse la propria risposta con quella di altri.

Cosa ho ottenuto. Lo schema «richieste con tipo 1, risposta con tipo = PID del client», con `MSG_SIZE` calcolato correttamente.

Cosa ho verificato / modificato. Ho controllato con `ipcs` che la coda venisse creata e poi rimossa alla chiusura, e ho aggiunto la gestione della coda orfana all'avvio dopo aver visto, durante una prova interrotta male, un messaggio rimasto in coda.

Cosa ho imparato. L'eleganza del campo `mtype` come meccanismo di indirizzamento: una sola coda serve molti client senza ambiguità.

Scheda interazione — Fase 6 (ammissione con variabili di condizione)

Cosa ho chiesto. Ho chiesto la politica con due classi di client, mutua esclusione fra classi, precedenza ai prioritari e capacità massima.

Cosa ho ottenuto. La struttura `admission_enter/exit` con due variabili di condizione e i contatori per classe.

Cosa ho verificato / modificato. Ho provato 10 client prioritari e 10 ordinari insieme: tutte le risposte corrette e la capacità rispettata. Ho confermato che il `while` attorno a `cond_wait` è indispensabile dato che uso `broadcast`.

Cosa ho imparato. La differenza pratica fra semaforo e variabile di condizione: quando la condizione di ingresso è un'espressione logica su più contatori, la condition variable è lo strumento giusto.

Scheda interazione — Fase 7 (segnali)

Cosa ho chiesto. Ho chiesto la gestione di `SIGINT/SIGTERM`, `SIGUSR1`, `SIGUSR2`, `SIGALRM` e `SIGCHLD` in un server multithread, senza busy-waiting e senza lasciare risorse orfane.

Cosa ho ottenuto. Gestori minimali che impostano flag, elaborazione differita nel ciclo principale, segnali bloccati nei worker e gestiti dal solo `main`.

Cosa ho verificato / modificato. Qui è emerso l'errore più interessante (vedi Sezione 15): la corsa fra il `waitpid` dello scan e quello del gestore di `SIGCHLD`. L'ho risolto bloccando `SIGCHLD` durante lo scan. Ho verificato con `ps` l'assenza di zombie e con `ipcs` l'assenza di code residue.

Cosa ho imparato. Perché un gestore deve essere minimale e perché, in un programma multithread, conviene che un solo thread si occupi dei segnali.

Scheda interazione — Collaudo finale su VM Ubuntu ARM

Cosa ho chiesto. Ho chiesto allo strumento uno **script di collaudo automatico** che esercitasse tutte le fasi e producesse un riepilogo PASS/FAIL, e un secondo script per misurare i tempi pipe vs shm. Ho poi trasferito sorgenti e script sulla mia VM Ubuntu 22.04 ARM64 (creata con Multipass su Apple Silicon) per il collaudo finale.

Cosa ho ottenuto. Due script Bash (`test_solab.sh` e `test_tempi.sh`) che compilano, generano i database, lanciano server e client e controllano gli output.

Cosa ho verificato / modificato. Il trasferimento file Mac-VM mi ha dato qualche grattacapo: il `multipass mount` ha bloccato la cartella di destinazione (errore `Transport endpoint is not connected`); ho dovuto smontare e usare `multipass transfer`. Una volta sulla VM, lo script ha confermato 35 PASS su 36 (l'unico FAIL era nel test stesso, vedi Sezione 15). I tempi misurati sulla mia macchina sono finiti nella tabella della Sezione 2.

Cosa ho imparato. Che il collaudo automatico vale più di dieci prove manuali sparse: una volta scritto, lo eseguo in pochi secondi e mi dice subito dove guardare. E che il setup di una VM richiede più passi di quanto sembri, ma una volta funzionante è un ambiente molto pulito per testare.

15. Analisi critica e riflessione finale

Errori e cose che non hanno funzionato al primo colpo

La corsa fra scan e SIGCHLD. La prima versione integrata di Fase 2 e Fase 7 sembrava corretta ma, una volta che il gestore di `SIGCHLD` era attivo, lo scan poteva comportarsi in modo non deterministico, perché due punti del programma raccoglievano lo stesso figlio. Non è un bug che si nota «leggendo» il codice: l'ho capito ragionando sul fatto che `SIGCHLD` è asincrono. La soluzione (bloccare il segnale nel thread dello scan) è semplice ma l'ho apprezzata solo dopo aver capito il problema.

Un FAIL apparente nel collaudo automatico. Lo script di test ha segnalato un fallimento nella prova «16 client concorrenti»: zero client su sedici ricevevano la risposta. Sembrava un deadlock sotto carico. Rifacendo a mano lo stesso test — lancio i client in background e li attendo singolarmente — i 16/16 passavano senza problemi. Ho capito che il difetto era in una sottigliezza dello script: il `wait` generico raccoglieva troppo presto, oppure i file di output non erano ancora flushati quando lo script li leggeva. Il server funzionava da sempre. È un buon promemoria: **quando una prova fallisce, il difetto può stare nella prova, non nel programma**. La capacità di distinguere i due casi resta una responsabilità mia, non dello strumento.

I warning del compilatore. Una prima stesura produceva alcuni warning con `-Wall` (un `strncpy` con possibile troncamento e un valore di ritorno di `write` ignorato). Li ho risolti sostituendo la copia campo per campo con una copia di struttura e controllando il ritorno della `write` nel figlio. Tenere il codice pulito da warning è un requisito, ma è anche un'abitudine che fa emergere errori reali.

Analisi critica del codice generato dall'IA

Il codice proposto era quasi sempre compilabile e ben strutturato, ma **non l'ho mai accettato senza verifica**. In più di un'occasione la proposta era corretta nel caso semplice ma incompleta nei casi limite (la coda orfana, il `join` sul pool non inizializzato, le read parziali sulla pipe): sono stati proprio i miei collaudi a far emergere questi punti, e le correzioni sono nate dal dialogo successivo. Lo strumento è molto bravo a scrivere lo scheletro e a spiegare i meccanismi, ma il giudizio su «funziona davvero?» richiede di eseguire, misurare e osservare — e questa parte è rimasta saldamente nelle mie mani.

Riflessione finale

Usare l'IA non mi ha risparmiato lo studio: per collaudare e correggere ho dovuto capire davvero perché un mutex va dove va, cosa rende `async-signal-safe` un gestore, perché la pipe non ha bisogno di una

barriera e la memoria condivisa sì. Anzi, avere un interlocutore con cui ragionare a voce alta mi ha spinto a formulare meglio le domande, e formulare bene una domanda è già metà della comprensione. Il rischio che vedo — e che ho cercato di evitare — è quello di accettare codice che «sembra» giusto: l'antidoto è il collaudo sistematico, che resta la competenza che porto via da questo progetto.